4. Design a URL Shortening Service

0) Requirements & Sizing

Functional

- Shorten long URLs, support custom aliases, link expiration, and disabling
- **302 redirects** (analytics-friendly) with very low latency
- Click analytics: total clicks, uniques, referrer/UA/geo, time series
- Admin: update/transfer ownership, bulk create, abuse reporting, preview/expand

Non-functional

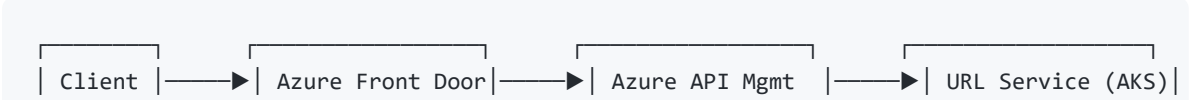
Metric	Target
Scale	~100M creates/day → ~1.16k writes/sec avg
Read:Write ratio	100:1 → ~116k reads/sec avg
Peak design	10× → **~1M RPS reads**
p99 redirect latency	< 20–30 ms from edge
Availability	Four 9s (99.99%) on read path
Global	Multi-region active-active, no SPOF

Capacity

```
7-char Base62 → 62⁷ ≈ 3.52 trillion codes
At 100M/day → ~96 years of headroom (future-proof)
```

1) High-Level Architecture

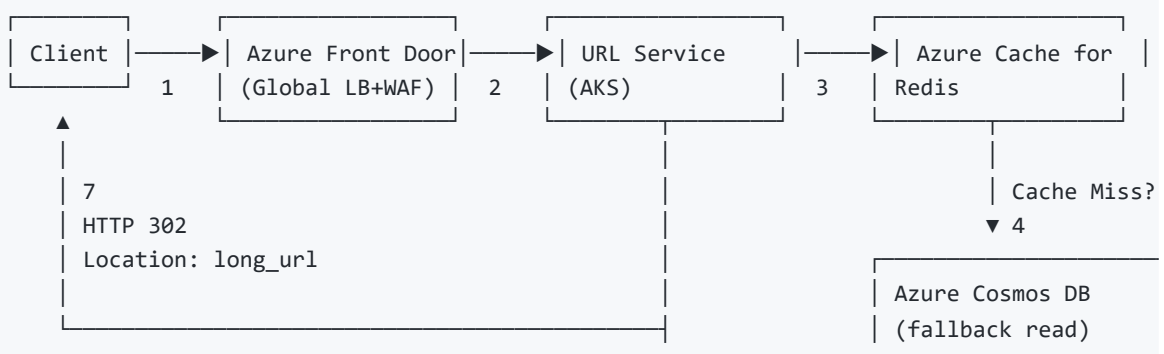
Write Path (Create Short URL)

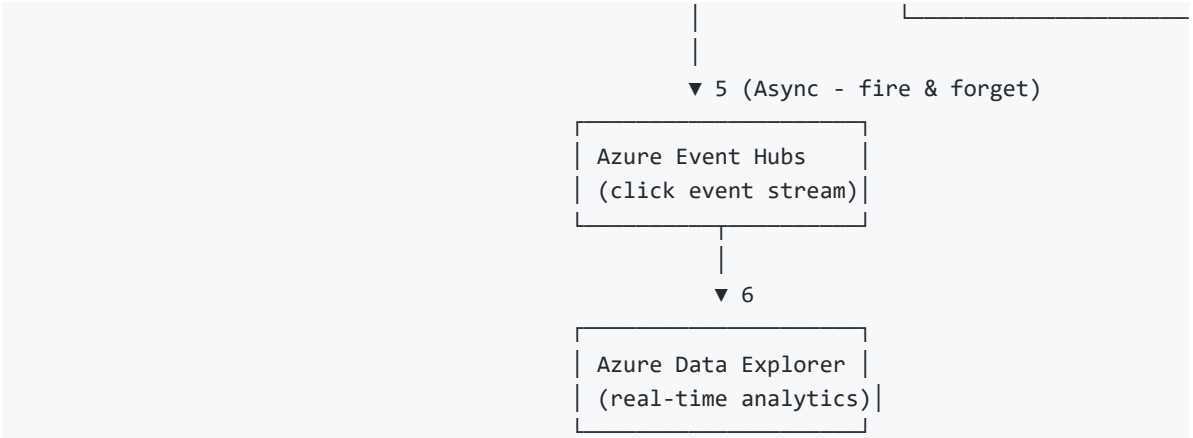




Step	From	To	Action	Sync/Async
1	Client	Azure Front Door	HTTPS request with long URL	SYNC
2	Front Door	API Management	Route to regional backend, WAF check	SYNC
3	API Management	URL Service (AKS)	Validate auth, apply rate limits	SYNC
4	URL Service	Key Generation Svc	Get next available short code from batch	SYNC
5	URL Service	Cosmos DB	Write mapping `{short_code → long_url}`	SYNC
6	Cosmos DB	Azure Functions	Change feed triggers indexing & analytics	ASYNC

Read Path (Redirect with Analytics)





Step	From	To	Action	Sync/Async
1	Client	Azure Front Door	GET `short.ly/abc123`	SYNC
2	Front Door	URL Service (AKS)	Route to nearest region	SYNC
3	URL Service	Azure Cache for Redis	Lookup `short_code` in cache	SYNC
4	Redis (miss)	Cosmos DB	Point read by `short_code`	SYNC
5	URL Service	Event Hubs	Emit click event `{code, ip, ua, ts}`	**ASYNC**
6	Event Hubs	Azure Data Explorer	Stream processing → dashboards	ASYNC
7	URL Service	Client	Return `HTTP 302 Location: long_url`	SYNC

> **Key insight**: Step 5 is ASYNC (fire-and-forget). Redirect returns immediately; analytics processed in background.

Components Summary

Component	Azure Service	Purpose
Global Entry	**Azure Front Door**	Anycast DNS, global load balancing, WAF

API Gateway	**Azure API Management**	Authentication, rate limiting, quotas
Compute	**Azure Kubernetes Service (AKS)**	Run URL service pods
Hot Cache	**Azure Cache for Redis**	~95-99% cache hit rate
Database	**Azure Cosmos DB**	Persistent KV store, global replication
Event Stream	**Azure Event Hubs**	Click events at high throughput
Analytics	**Azure Data Explorer (ADX)**	Sub-second aggregation queries
Serverless	**Azure Functions**	Change feed processing

API Design

Core APIs

Method	Endpoint	Description
`POST`	`/api/v1/shorten`	Create a short URL
`GET`	`/{shortCode}`	Redirect to original URL (302)
`GET`	`/api/v1/urls/{shortCode}`	Get URL metadata
`DELETE`	`/api/v1/urls/{shortCode}`	Delete a short URL

Request/Response Examples

Create Short URL:

```
POST /api/v1/shorten
{
  "long_url": "https://example.com/very/long/path",
  "custom_alias": "my-link",    // optional
  "expiry_days": 30             // optional
}

Response: 201 Created
{
  "short_url": "https://short.ly/abc123",
  "short_code": "abc123",
```

```
    "expires_at": "2025-04-15T00:00:00Z"
  }
```

Redirect:

```
GET /abc123
Response: 302 Found
Location: https://example.com/very/long/path
```

Analytics APIs

Method	Endpoint	Description
`GET`	`/api/v1/urls/{shortCode}/stats`	Get click analytics
`GET`	`/api/v1/urls/{shortCode}/stats?range=7d`	Get time-range analytics

2) ID Generation

✅ Recommended: Pre-generated Batches

```
Key Service pre-generates Base62 keys by ranges
  ├── Batch [aaa...azz] → API Pod 1
  ├── Batch [baa...bzz] → API Pod 2
  └── Batch [caa...czz] → API Pod 3

On create: pop from local pool
If empty: atomically fetch next batch

✅ No coordination on hot path
✅ No collisions
✅ No SPOF
```

Alternative: Snowflake ID

```
┌──────────────┬────────────┬────────────┬────────────┐
│  Timestamp   │ Datacenter │  Machine   │ Sequence │
│  (41 bits)   │  (5 bits)  │  (5 bits)  │ (12 bits)│
└──────────────┴────────────┴────────────┴────────────┘

→ Base62 encode → short code

✅ Distributed, no coordination
⚠️ Set region bits to scatter across partitions
```

❌ Counter: Reject

Single bottleneck + SPOF. Not scalable.

Custom Aliases

- Conditional write: "create if not exists" (Cosmos `If-None-Match`)
- Reserved terms list + profanity filter (enterprise requirement)

---

3) Data Model (Cosmos DB)

```
{
  "id": "7HgT2mA",              // short_code (partition key)
  "long_url": "https://...",
  "owner_id": "aad:...",
  "created_at": "2026-01-27T00:00:00Z",
  "expires_at": "2027-01-27T00:00:00Z",
  "status": "active|disabled|deleted",
  "flags": {
    "allow_params": true,
    "safe": true
  }
}
```

- **TTL** on `expires_at` for auto-delete
- **Change feed** → secondary container indexed by `(owner_id, created_at)` for "my links" view

---

3.1) Storage Options: Table Storage vs Cosmos DB vs Blob

Azure Storage Account as KV Store?

Service	Pattern	Best For
Table Storage	PartitionKey + RowKey → value	Cheap, simple KV, moderate throughput
Blob Storage	blob name → blob content	Large objects, not high-frequency KV
Cosmos DB	id → document	Low latency, global, high throughput

Comparison

Aspect	Table Storage	Cosmos DB
Latency	~10-50ms	**<10ms** (single-digit)
Throughput	≤20K ops/sec per table	**10M+ ops/sec**
Global distribution	❌	✅
Automatic indexing	❌ (PK+RK only)	✅ (all properties)
Cost	**Cheapest**	Higher
Query patterns	PK+RK point queries only	Rich queries

When to Use What

Use Table Storage if:

- Budget is tight
- Queries are only `PartitionKey + RowKey → value`
- No global distribution needed
- Moderate throughput (≤20K ops/sec)

Use Cosmos DB if:

- Need predictable low latency (<10ms)
- Large scale or global distribution
- High throughput requirements
- Need advanced queries or change feed

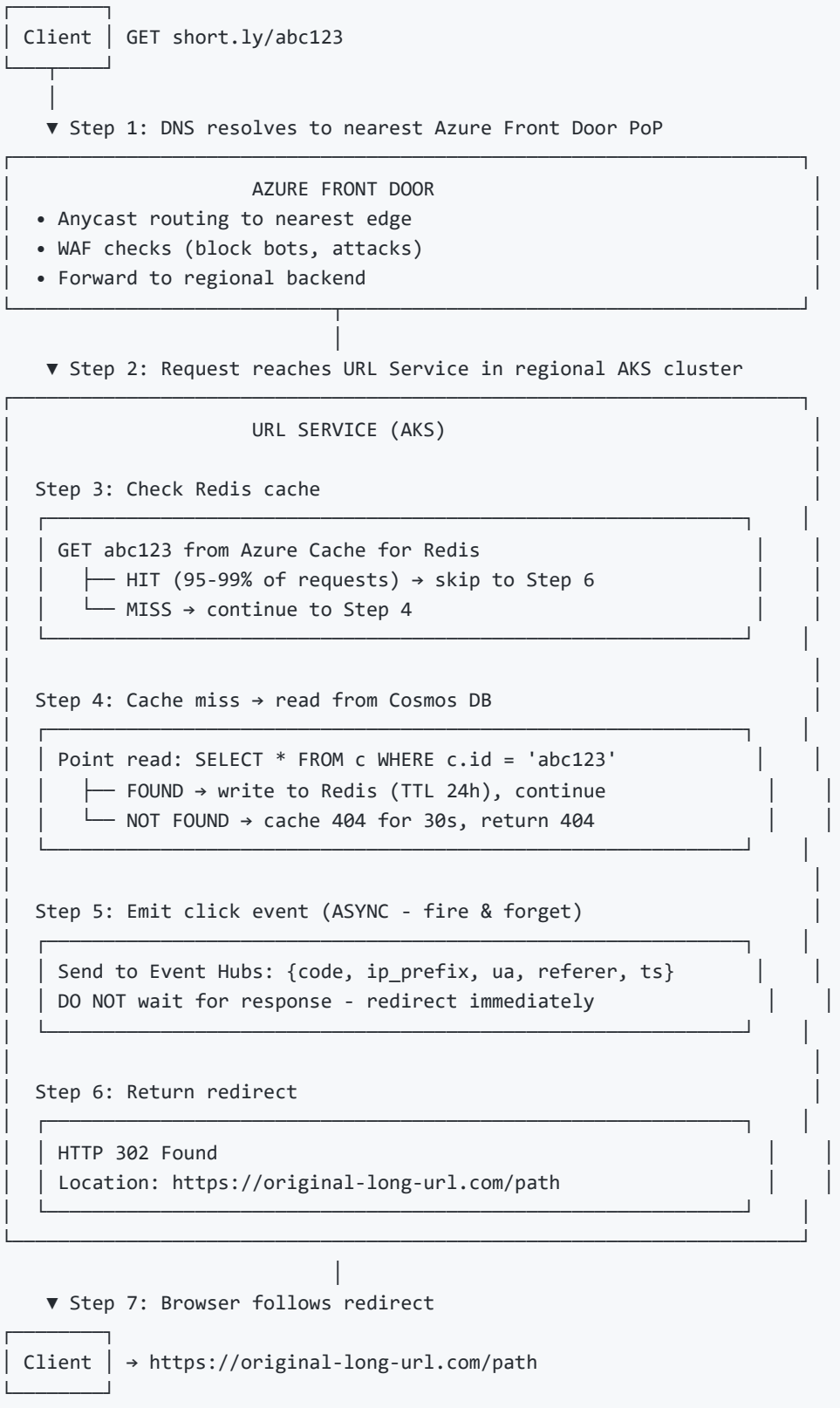
For URL Shortener: Cosmos DB

Why?

- Point reads by `short_code` = perfect KV pattern
- Need low latency for redirects (<30ms p99)
- Change feed powers analytics pipeline
- Multi-region active-active required

---

4) Read Path Detail



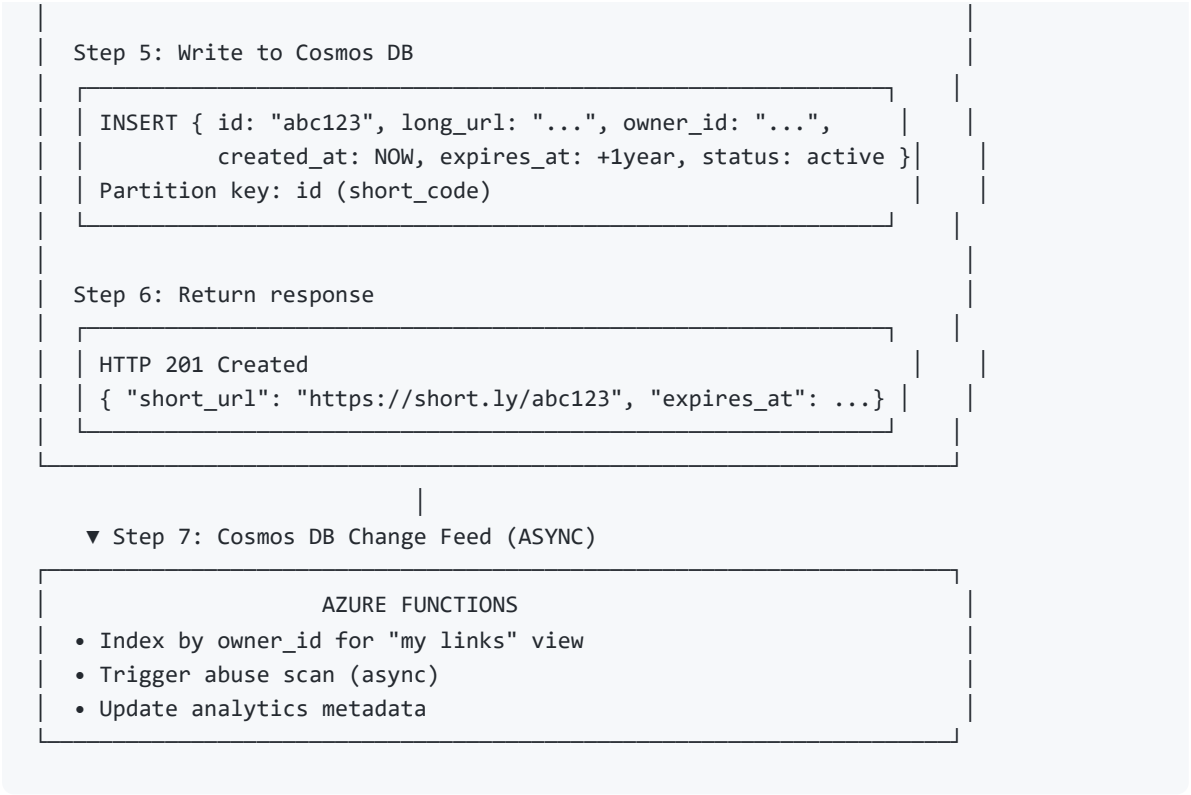
Cache Strategy:

Setting	Value	Why
TTL	24h-7d	URLs rarely change

Early refresh	Probabilistic	Prevent thundering herd on expiry
Eviction	LRU	Keep hot URLs, evict cold
Negative cache	30s for 404s	Block brute-force scanning
Dogpile protection	Single-flight per key	One DB call per miss

5) Write Path Detail





---

6) CDN vs Analytics (Critical Trade-off)

Choice	Performance	Analytics
CDN caches redirects	✅ Fastest	❌ Can't count every click
No CDN, use LB+Redis	✅ Fast	✅ Count every click

> ⚠️ **If every click must be counted, do NOT cache redirects at CDN.** Use Front Door as L7 LB + WAF only, serve from app/Redis.

Options:

- Per-link toggle: cache 301 for very hot links that don't need analytics
- Edge workers: increment counter + issue 302 (but beware log loss under cache hits)

---

7) Analytics (Clicks, Uniques)

On each redirect, emit to Event Hubs:

```
{ "ts": "...", "code": "7HgT2mA", "ip_prefix": "1.2.3.0/24",
```

```
    "ua": "...", "referer": "...", "region": "westus" }
```

Streaming jobs:

- Real-time counters in Redis or ADX materialized views
- **HyperLogLog** per (code, day) for memory-efficient unique counts
- Persist detailed events to ADX; roll older data to cold tier

---

8) Multi-Region & DR

```
Active-active: East US, West Europe, Central India
  │
  ├── Front Door anycast → nearest healthy region
  ├── Cosmos DB multi-write (last-writer-wins, safe for rare writes per key)
  ├── Redis: independent per region (no cross-region replication)
  └── Event Hubs: per-region with geo-replication to centralized ADX
```

Key allocation: Add region prefix in Snowflake or disjoint key ranges to avoid collisions.

---

9) Security & Governance

Concern	Solution
AuthN/Z	AAD for UI, API keys with scopes
WAF	Front Door WAF for bot signatures
Ownership	Require two owners or group; profanity/reserved checks
PII	Truncate IPs (/24), hash UA for uniques
GDPR	Deletion pipeline for click rows
SSRF	Block private IPs, enforce public host allow-list

---

10) Failure Modes

Failure	Mitigation
Redis down	Fall back to DB; shed load with token buckets; 503 for tail

DB region loss	Multi-region Cosmos continues; Front Door routes to healthy region
Key service outage	Writes degrade; reads unaffected; keep local key buffer for hours
Analytics lag	Decouple via Event Hubs; redirects continue; backpressure to consumers

---

11) Cost Estimates

Component	Sizing
Redis	Hot set ~20M keys × 300B = ~6GB + overhead per shard
Cosmos DB	Point reads ~1 RU; size for peak writes; autoscale
Event Hubs	Throughput units for peak click RPS; retention ≤7 days

---

12) 60-Second Explanation

> "It's a **read-heavy key-value system**. I generate 7-char Base62 codes using **pre-allocated batches** to avoid coordination. The mapping lives in **Cosmos DB** (globally distributed) fronted by a large **Redis cache** (~99% hit rate).
>
> Redirects return **302** so I can count clicks. Every hit emits a tiny event to **Event Hubs**; stream jobs aggregate into **ADX** for real-time dashboards.
>
> The whole thing is **active-active** across regions behind **Front Door**. We prevent SPOFs, use conditional writes for vanity conflicts, TTL/negative caches to stop brute forcing, and change feed for side effects.
>
> This keeps **p99 < 30ms** and scales to **~1M RPS peaks** while preserving accurate analytics."

Technologies Deep Dive

Technology	Role	Reference
Cosmos DB	Distributed KV store	Tech Guide
Redis	Hot cache, counters	Tech Guide

Kafka/Event Hubs	Click event streaming	Tech Guide
HyperLogLog	Unique click counts	Tech Guide
Consistent Hashing	Shard distribution	Tech Guide

Appendix: Base62 Encoding

```
String alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";

String encode(long n) {
    StringBuilder s = new StringBuilder();
    do {
        s.append(alphabet.charAt((int)(n % 62)));
        n /= 62;
    } while (n > 0);
    return s.reverse().toString();
}

long decode(String s) {
    long n = 0;
    for (char c : s.toCharArray()) {
        n = n * 62 + alphabet.indexOf(c);
    }
    return n;
}
```